Java Backend Architecture

Interview Series

Chapter 19.1

CI/CD Pipeline Design

50+ Interview Q&A	Code Examples	Architecture Diagrams	Advanced Topics

Chapter 19.1 – CI/CD Pipeline Design

Overview

A CI/CD pipeline automates the path from developer commit to production deployment, ensuring Java applications are built, tested, scanned, and deployed reliably and repeatably. GitHub Actions, Jenkins, and GitLab CI are the dominant platforms. Understanding pipeline stages, parallelism, artifact management, environment promotion, and OIDC-based keyless cloud authentication is essential for senior Java backend engineers.

Key Definitions

Term	Definition
CI (Continuous Integration)	Automatically build and test every commit. Catch integration failures early.
CD (Continuous Delivery)	Every passing build is deployable. Deployment to production requires manual approval.
CD (Continuous Deployment)	Every passing build is automatically deployed to production without manual gates.
GitHub Actions	GitHub-native CI/CD. Workflows defined in .github/workflows/*.yml.
Jenkins	Open-source CI/CD server. Declarative Pipelines in Jenkinsfile. Large plugin ecosystem.
GitLab CI	GitLab-native CI/CD. Stages/jobs in .gitlab-ci.yml. Built-in container registry.
OIDC token	OpenID Connect token for keyless cloud auth. GitHub Actions can assume AWS IAM roles without long-lived credentials.
Matrix build	Run same job across multiple parameter combinations (Java 17+21, OS linux+mac).
Artifact	Build output (JAR, Docker image) passed between pipeline stages or stored for deployment.
Environment gate	Manual approval required before deploying to production. Defined via GitHub Environments.

Diagram 1: Java CI/CD Pipeline Stages

```
Developer Push
     |
     v
[1] Checkout + Cache (Maven/Gradle deps)
     |
     v
[2] Build (mvn package / gradle build) -- compile + unit tests
     |
     v
[3] Code Quality (SonarQube / SpotBugs / Checkstyle)
     |
     v
[4] Integration Tests (Testcontainers / Spring Test)
     |
     v
[5] Security Scan (Snyk / Dependabot / Trivy image scan)
     |
     v
[6] Build Docker Image + Push to Registry
     |
     v
[7] Deploy to Staging (auto)
     |
     v
[8] E2E Tests on Staging
     |
     v
[9] Deploy to Production (manual approval gate)
```

Diagram 2: GitHub Actions OIDC Keyless Auth Flow

```
GitHub Actions Job
     |
     | 1. Request OIDC token from GitHub
     v
GitHub OIDC Provider (token.actions.githubusercontent.com)
     |
     | 2. Issue JWT with claims (repo, branch, workflow)
     v
GitHub Actions Job
     |
     | 3. AssumeRoleWithWebIdentity(token)
     v
AWS STS
     |
     | 4. Verify JWT signature against GitHub JWKS
     | 5. Check trust policy conditions (repo, branch)
     v
Temporary AWS credentials (15 min TTL)
     |
     v
Push to ECR / Deploy to EKS (no long-lived AWS_SECRET_ACCESS_KEY)
```

Code Examples

GitHub Actions -- Full Java CI Workflow:

```
name: Java CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        java: [17, 21]          # test on multiple JDK versions
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '${{ matrix.java }}', distribution: temurin }
      - uses: actions/cache@v4
        with:
          path: ~/.m2/repository
          key: maven-${{ hashFiles('**/pom.xml') }}
      - run: mvn -B package --no-transfer-progress
      - run: mvn sonar:sonar -Dsonar.token=${{ secrets.SONAR_TOKEN }}

  docker-build-push:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    permissions:
      id-token: write          # required for OIDC
      contents: read
    runs-on: ubuntu-latest
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/github-actions-ecr
          aws-region: us-east-1
      - run: aws ecr get-login-password | docker login --username AWS --password-stdin $ECR_URI
      - run: docker build -t $ECR_URI/java-app:${{ github.sha }} .
      - run: docker push $ECR_URI/java-app:${{ github.sha }}

  deploy-production:
    needs: docker-build-push
    environment: production      # requires manual approval
    runs-on: ubuntu-latest
    steps:
      - run: kubectl set image deployment/java-app app=$ECR_URI/java-app:${{ github.sha }}
```

Jenkins Declarative Pipeline:

```
pipeline {
    agent { docker { image 'maven:3.9-eclipse-temurin-21' } }

    stages {
        stage('Build') {
            steps { sh 'mvn clean package -DskipTests' }
        }
        stage('Test') {
            parallel {
                stage('Unit')        { steps { sh 'mvn test' } }
                stage('SpotBugs')    { steps { sh 'mvn spotbugs:check' } }
                stage('SonarQube')   {
                    steps {
                        withSonarQubeEnv('SonarQube') { sh 'mvn sonar:sonar' }
```

```
                    }
                }
            }
        }
        stage('Docker Build') {
            steps {
                sh "docker build -t java-app:${env.BUILD_NUMBER} ."
                sh "docker push registry/java-app:${env.BUILD_NUMBER}"
            }
        }
        stage('Deploy Prod') {
            when { branch 'main' }
            input { message 'Deploy to production?' }
            steps { sh "kubectl set image deployment/java-app app=registry/java-app:${env.BUILD_NUMBER}" }
        }
    }
    post {
        failure { slackSend(message: "Build FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}") }
    }
}
```

Interview Q&A – CI/CD Pipeline Design

Q1. What is the difference between Continuous Delivery and Continuous Deployment?

Continuous Delivery: every passing build is deployable, but deployment to production requires a manual approval gate. Team can deploy on demand. Continuous Deployment: every passing build is automatically deployed to production -- no human gate. Requires extremely high test coverage and confidence in automated checks. Most teams use Continuous Delivery with manual approval for production and auto-deploy to staging/QA.

Q2. How does OIDC keyless authentication work in GitHub Actions?

GitHub Actions requests a JWT OIDC token from GitHub's OIDC provider. The token contains claims: repository, branch, workflow, environment. The Actions job calls AssumeRoleWithWebIdentity (AWS) or Workload Identity Federation (GCP) with this token. Cloud provider verifies token signature against GitHub's JWKS endpoint and checks trust policy conditions (allowed repo/branch). Returns temporary credentials with TTL. Eliminates long-lived AWS_SECRET_ACCESS_KEY in repository secrets.

Q3. How do you implement parallel testing in GitHub Actions?

Matrix strategy: matrix: { java: [17, 21], os: [ubuntu-latest, macos-latest] } creates 4 parallel jobs. Within a job: use parallel keyword (Jenkins) or split test suites. For Gradle: --parallel flag. For Maven: maven-surefire-plugin with forkCount. Upload test results as artifacts: actions/upload-artifact for JUnit XML, then aggregate in a summary job. Parallel testing reduces CI time from O(n) to O(n/threads).

Q4. What is a reusable workflow in GitHub Actions?

workflow_call trigger allows a workflow to be called from another workflow like a function. Define inputs (java-version, environment) and secrets. Caller: uses: ./.github/workflows/deploy.yml with inputs and secrets. Enables DRY pipeline design: shared build, test, deploy workflows across multiple Java microservice repositories. Combine with workflow_dispatch for manual triggers and reuse across org repositories.

Q5. How do you manage secrets in CI/CD for Java deployments?

GitHub Secrets: encrypted at rest, exposed as environment variables in workflow. Never print secrets (GitHub Actions masks known secrets in logs). Prefer OIDC over static secrets for cloud access. For runtime secrets (DB password, API keys): use AWS Secrets Manager / GCP Secret Manager injected at deployment via Kubernetes External Secrets Operator, not baked into Docker images. Rotate secrets regularly, audit access.

Q6. What is GitLab CI's advantage for containerised Java builds?

GitLab CI has built-in Container Registry, eliminating external registry dependency. Runners with Docker-in-Docker (dind) or Kaniko for image builds. rules: (if/changes) conditional job execution -- skip backend CI if only frontend changed. cache: with key based on lock files. artifacts: pass JARs/Docker tags between stages. environments: with auto_stop_in for ephemeral review environments. All on same platform, no external integrations needed.

Q7. How do you implement blue-green deployment in a CI/CD pipeline?

1) Build and push new Docker image (green). 2) Deploy green deployment (scaled to 0 initially). 3) Run smoke tests against green directly. 4) Shift 100% traffic to green (update LB target group or K8s Service selector). 5) Monitor error rate for 5-10 minutes. 6) Delete or scale down blue deployment. Rollback: repoint LB back to blue. In Kubernetes: kubectl set selector on Service to switch between blue and green Deployments.

Q8. What is the purpose of caching dependencies in CI/CD?

Maven/Gradle downloads dependencies on every build from Maven Central -- slow (1-3 min) and fragile (network failure). Caching ~/.m2/repository or ~/.gradle/caches with a key based on pom.xml/build.gradle hash: first run downloads and caches, subsequent runs restore from cache (seconds). GitHub Actions cache: actions/cache@v4. Cache invalidates automatically when dependency files change. Reduces Java build time by 60-80%.

Q9. How do you prevent deploying to production on failed security scans?

Add security scan as a required CI stage: Snyk (SCA for dependency vulnerabilities), Trivy (container image scan), SpotBugs/SonarQube (SAST). Gate configuration: if scan returns CRITICAL vulnerabilities, fail the pipeline (non-zero exit code). GitHub: required status checks on branch protection rules -- merge blocked if scan fails. Set severity thresholds: fail on CRITICAL, warn on HIGH for Java projects.

Q10. What is a shared library in Jenkins and why is it useful?

Shared libraries in Jenkins allow Groovy code to be shared across multiple Pipelines: vars/ directory for global variables/functions, src/ for classes. Usage: @Library('shared-library') import com.company.pipeline.*. Enables: standardised build functions across

Java microservices (def buildJavaApp(String version) {...}), common notification logic, security scanning steps. Changes to library apply to all pipelines on next run.